Supervised Machine Learning for Software Effort and Cost Estimation Using the COCOMO Approach

Group Name: Ranković 4
Alexandra Antonica (u593275)
Pien Jansen (u559615)
Barbara Koch (u873859)
Kaloyan Milchin (u656029)
Prakhar Tiwari (u801772)

Abstract

Accurate software effort and cost estimation are crucial for project management in the software industry. This study uses the revised version of the COCOMO2000 dataset to assess the performance of the XGBoost, Support Vector Machine (SVM), and Logistic Regression models. Our detailed evaluation demonstrates that SVM and XGBoost outperform Logistic Regression on the evaluation metrics of accuracy and recall. As machine learning techniques emerge as useful tools for dealing with software estimation issues, our findings can help companies optimize resource allocation, reduce project risks, and enhance overall project planning and execution efficiency.

Keywords: Software Estimation; COCOMO; Logistic Regression; XGBoost; Classification

Introduction

Software effort and cost estimation have become an important step in the software development life cycle, as well as in managing the project cost, time and quality (Kumar et al., 2023). Consequently, we need accurate estimations to ensure the project's success and prevent risks. With software effort estimation presenting challenges in the software industry, machine learning methods are emerging as valuable tools to address this issue (Kumar et al., 2023). Therefore, in this research proposal, we present an investigation into the comparative performance of specific machine learning algorithms in classifying the accuracy of software effort and cost estimation using the Constructive Cost Model (COCOMO) approach.

As the complexity and scale of software projects have expanded, the demand for precise software estimation has surged notably over recent decades (Mahmood et al., 2021b). Businesses are increasingly in pursuit of highly accurate projections concerning the delivery timelines and financial requirements for their software projects, aiming to provide their clients with reliable commitments (Rao et al., 2022). The challenge of achieving precise software estimation lies at the heart of project success, and by extension, the broader success of companies engaged in software development. This underscores the critical need for effective estimation methodologies. For the past years, various models and methodologies have been developed to tackle this challenge such as COCOMO (Bardsiri & Dorosti, 2016).

However, with the advancements in fields like machine learning, we have been able to improve the software estimation accuracy even more. Techniques such as neural networks, regression and categorical analysis have been applied to estimation tasks, yielding predictions that are more aligned with the actual outcomes of software projects (Alauthman et al., 2023).

The integration of traditional estimation methods with machine learning technologies marks a significant advancement in the field of accurate software project estimation (BaniMustafa, 2018). This development not only improves the reliability of project predictions but also equips project managers and stakeholders with valuable insights.

The motivation for our research stems from both scientific and societal relevance. In the domain of software engineering, diverse methods of effort and cost estimation exist, and, while largely complex, their accuracy remains a crucial area for improvement (Mahmood et al., 2021). Ideally, a model should capture the full range of complexities and variations inherent in software development projects (Alauthman et al., 2023). Our research aims to address this gap, by combining the strengths of XGBoost and Support Vector Machine models. Specifically, our focus is on offering more reliable predictions for software companies and their clients, essential for contract negotiations and project execution. Our proposal provides a novel approach to software effort and cost estimation, due to our methodologies, which are based on the chosen machine learning algorithms.

Based on the stated research gap the following research question was formed: To what extent do Machine Learning algorithms, specifically XGBoost and Support Vector Machine, compare to Logistic Regression in classifying the accuracy of software effort and cost estimation using the COCOMO approach?

RQ1: How do the performance of XGBoost, Support Vector Machine, and Logistic Regression algorithms compare when evaluated using the confusion matrix and accuracy metrics in this context?

RQ2: How does the incorporation of data augmentation techniques impact the accuracy and performance of XGBoost, Support Vector Machine, and Logistic Regression in this context?

Methodology

The research process of this study is illustrated in the flowchart in Figure 1. The main steps of the research process are further explained in this section and the Results section.

Dataset Description

We used the revised version of the COCOMO2000[1] dataset. Table 1 represents the overall description of the input features, while Table 2 captures descriptive statistics about them. A histogram representing the distribution of the target variable (i.e., actual effort, ACT_EFFORT) can be observed in Figure 2. The correlation between the features and the target can be visualized in the heatmap in Figure 3.

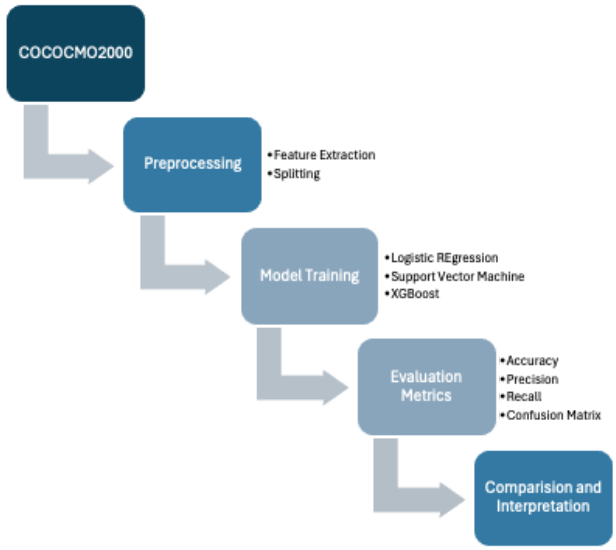


Figure 1: Flowchart of the Research Process

Table 1: Description of input variables

Name of the Feature	E	PEMi	Size KLOC
Original Type	Numerical	Numerical	Numerical
Description	The effort estimation to complete the software development	Product of 17 effort multipliers	The number of thousands of Lines of Code written during the software development.
Example	1.041	1.041	15

[1] available at the following repository: http://promise.site.uottawa.ca/SERepository/datasets-page.html

Table 2: Descriptive statistics of the data

	E	PEMi	Size KLOC	ACT_EFFORT
count	97.000	97.000	97.000	97.000
mean	1.076	1.546	104.565	610.204
std	0.045	1.040	148.326	1114.844
min	0.972	0.530	0.000	8.400
25%	1.041	1.028	20.000	67.700
50%	1.056	1.165	60.000	252.000
75%	1.136	1.808	137.000	599.000
max	1.167	7.409	980.000	8211.000

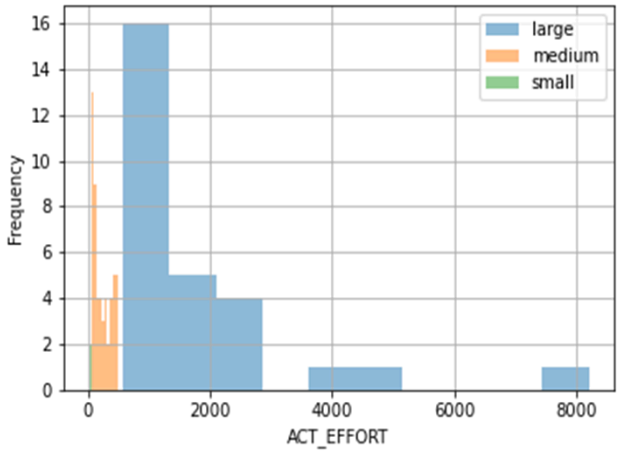


Figure 2: Distribution of ACT_EFFORT by Categorical Output

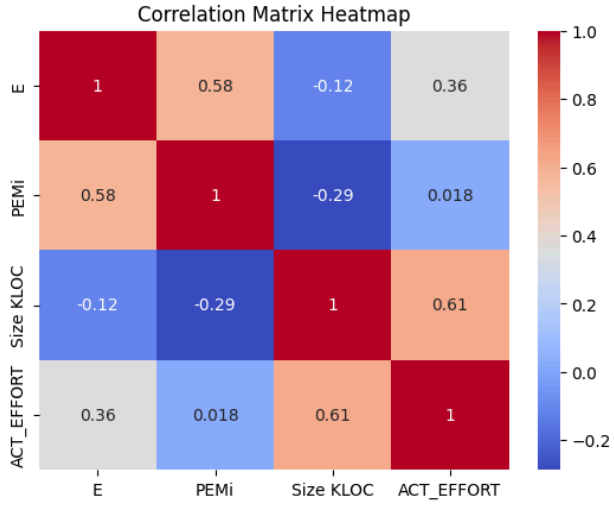


Figure 3: Correlation of the input variables to the target

Feature Extraction

The COCOMO2000 dataset initially consisted of 22 parameters that were divided into 2 groups - 5 scaling factors (prec, flex, resl, team, pmat) and 17 effort multiplier factors (rely, cplx, data, ruse, time, stor, pvol, acap, pcap, pcon, apex, plex, ltex, tool, sced, site, docu). After calibration, 3 of the input features were extracted: E (Effort), PEM (Personnel Effort Multipliers), and KLOC (Thousands of Lines of Code), as seen in Equations (1) to (4) below (Ranković et al., 2021). We will use the hold-out approach, splitting the data by 80% for training and 20% for testing. We will proceed further by using the K-fold cross-validation method in parallel to the hold-out approach for optimizing the model.

$$Effort = Ax[SIZE]^{E} \times \prod_{i=1}^{17} EM_i,$$

$$where\ E = B + 0.01 \times \sum_{j=1}^{5} SFjA = 2.94,\ B = 0.91 \quad (1),$$

$$Effort[PM] = 2.94 \times [SIZE]^{E} \times PEM_i,$$

$$where\ PEM_i = \prod_{i=1}^{17} EM_i \quad (2)$$

$$Time = C \times (Effort)^{F},$$

$$where\ F = D + 0.2 \times 0.01 \times \sum^{5} SFj,\ C = 3.67,\ D = 0.28 \quad (3)$$

$$People = Effort\ /\ Time \quad (4)$$

Machine Learning Models

For this research, we used Support Vector Machine (SVM) and XGBoost machine learning algorithms for the classification task on the COCOMO2000 dataset, with Logistic Regression as the baseline model. The description and motivation for using SVM and XGBoost are as follows:

(1) SVM is a supervised learning algorithm that finds a hyperplane in the feature space that maximizes the margin between the classes. SVM is a powerful algorithm for classification tasks, often exhibiting high accuracy and robustness to noise. It offers an alternative approach to ensemble methods and can be particularly suitable for problems with well-defined class boundaries. (Yusri et al., 2022)

(2) XGBoost is an advanced implementation of gradient boosting, a technique that iteratively builds decision trees, focusing on improving predictions for previously misclassified instances. It offers regularization techniques to prevent overfitting and is known for its speed and efficiency. Its regularization capabilities further enhance its performance in comparison to simpler models. (Yusri et al., 2022)

Evaluation Measures

For our research, we have chosen 3 performance measures: Confusion matrix, Accuracy, and Recall metrics. The motivation behind using them is as follows: The confusion matrix provides a detailed breakdown of the model's performance, allowing for the identification of specific types of errors (overestimation or underestimation) and potential areas for improvement. Accuracy (see Equation (5)) measures the overall proportion of correctly classified instances. It provides a general understanding of how well the model performs overall. Recall (see Equation (6)) measures the proportion of actual positive instances that are correctly identified by the model. It is important when it is critical to avoid missing true positives (Naser & Alavi, 2020), such as failing to identify potentially high-cost projects.

$$Accuracy = \frac{True\ Positives + True\ Negatives}{Positives + Negatives} \quad (5)$$

$$Recall = \frac{True\ Positives}{Positives} \quad (6)$$

Results

For this study, we used supervised machine learning algorithms XGBoost and Support Vector Machine (SVM) on the COCOMO2000 dataset. Their performance was compared to the baseline Logistic Regression model to classify Software Effort and cost estimation. Overall, both SVM and XGBoost models outperformed the baseline Logistic Regression on all of the selected evaluation metrics.

The violin plot in Figure 4 shows each model's predictions for the target variable ACT_EFFORT. It successfully employs kernel density estimation to represent the distribution, density, and range of projected values using the colors blue, orange, and green for Logistic Regression, SVM, and XGBoost, respectively. The logistic regression distribution appears symmetric with moderate variability in the predictions around the median. While the plot of SVM illustrates a broader and multi-modal distribution and shows significant peaks at various levels, a more narrow distribution can be seen in the plot for XGBoost, indicating a more concentrated prediction around the median.

Table 1 provides a numerical representation of the results, while Figure 5 illustrates the bar chart. The performance of various models is contrasted in Table 3 according to the evaluation metrics of recall and accuracy. In addition, we computed the confusion matrices for each of the implemented models as a visualization tool. The confusion matrices corresponding to XGBoost, SVM, and Logistic Regression are presented in heatmaps in Figure 6. The column-wise display of the Actual Classes (small, medium, and large) contrasts with the row-wise display of the Predicted Classes.

In this study, we used three performance metrics: confusion matrix, recall, and accuracy. We aligned our selection with the experimental setup of BaniMustafa (2018), who used accuracy and recall as evaluation metrics on several machine learning models for his study, including Logistic Regression. Accuracy is a measure of how well a model correctly classifies occurrences, providing a general assessment of its overall performance. Recall quantifies the fraction of true positive cases accurately detected by the model. Moreover, evaluating the review of Akumba et al.

(2023), we also decided on the confusion matrix as a result visualization tool that gives enough area for a discussion on the models' performance. The confusion matrix offers us a complete assessment of the models' performance, helping identify error kinds and prospective improvement areas.

Overall, the Support Vector Machine (SVM) and XGBoost outperformed Logistic Regression across test set accuracy and recall metrics. Specifically, SVM achieved an accuracy of 0.816 and a recall of 0.836, while XGBoost recorded an accuracy of 0.803 and a recall of 0.736. In contrast, Logistic Regression yielded an accuracy of 0.763 and a recall of 0.654. The SVM exhibited the most favorable outcomes based on the evaluation metrics.

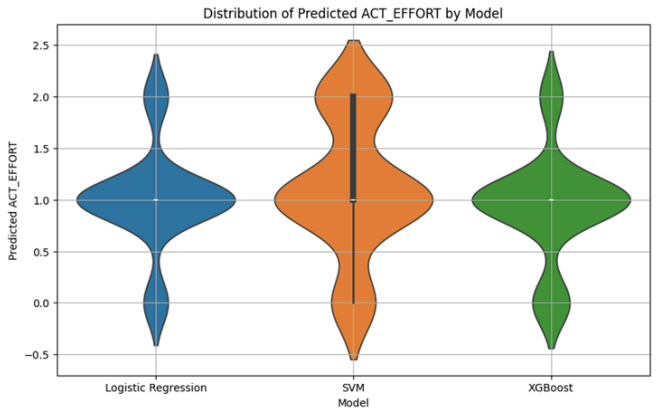


Figure 4: Violin plot of the distribution of predicted target variable by each model

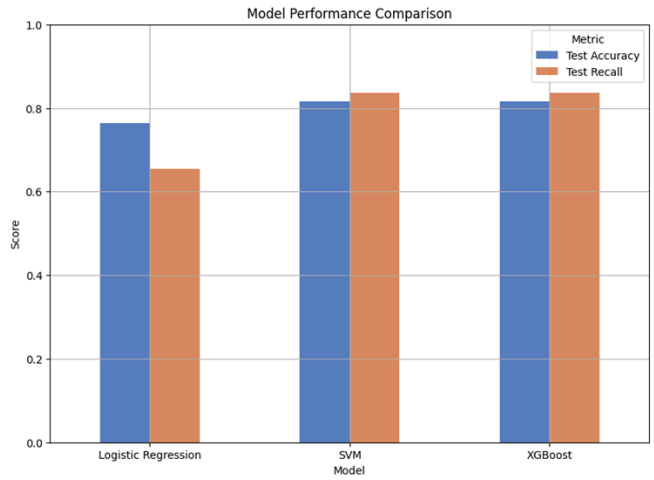
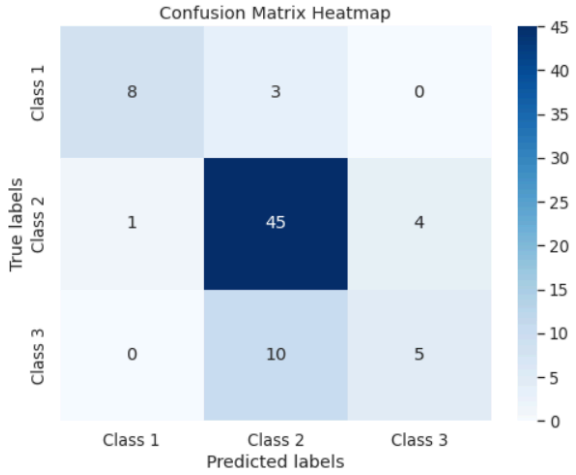


Figure 5: Bar chart of evaluation metrics for each machine learning model

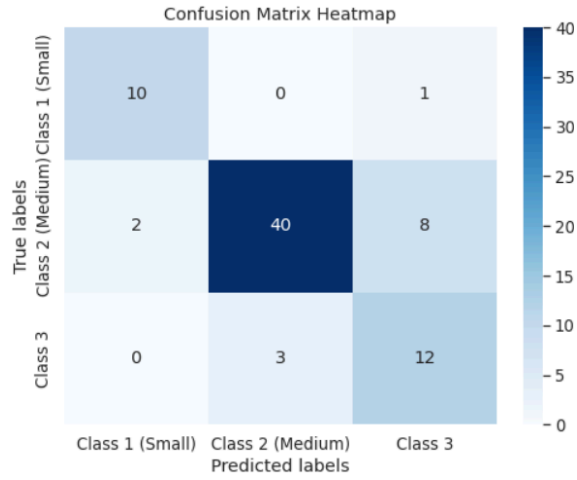
Table 3: Evaluation metrics on the test set for each machine learning model

		Evaluation Metric	
		Test set Accuracy	Test set Recall
Model	Logistic Regression	0.763	0.654
	Support Vector Machine	0.816	0.836
	XGBoost	0.803	0.736

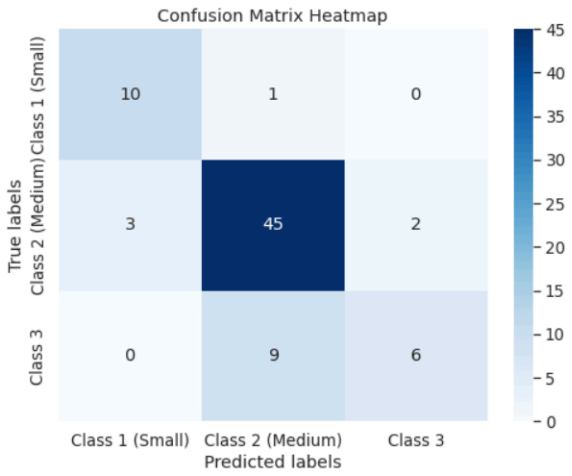
Figure 6: Heatmap for Confusion Matrix of a) Logistic Regression, b) Support Vector Machine and c) XGBoost



a. Logistic Regression Confusion Matrix



b. SVM Confusion Matrix



c. XGBoost Confusion Matrix

However, models are biased towards the majority class due to imbalance, thereby overfitting the model and misleading the metrics. This made data augmentation a crucial aspect for the models to be robust and to get interpretable results.

Regarding computational efficiency, Logistic Regression required approximately 0.74 seconds to complete the classification task. On the other hand, the SVM and XGBoost models required about 28 minutes and 2.35 seconds, and 77 minutes and 13.58 seconds, respectively, underscoring a significant increase in computation time for the more complex models.

Discussion

The correlation between 'Size KLOC' and 'ACT_EFFORT', which exhibited a strong positive correlation of 0.61, significantly influenced the model performance. This correlation supports the COCOMO model's assumption that larger projects generally necessitate greater effort, validating the effectiveness of SVM and XGBoost in capturing this relationship. These models are particularly good at leveraging such strong linear relationships to improve prediction accuracy, as evidenced by their superior performance metrics.

Conversely, the very low correlation between 'PEMi' and 'ACT_EFFORT' (0.018) revealed that effort multipliers, despite their intended role in reflecting project complexity and requirements, do not straightforwardly predict actual effort in project executions. This insight is critical as it shows why Logistic Regression, which relies more on linear assumptions, frequently misclassifies especially in instances involving complex interactions that the model cannot adequately capture. In contrast, SVM and XGBoost, with their capability to handle non-linear relationships and intricate model interactions—through kernel methods and ensemble tree-based approaches respectively—show marked improvement in classification accuracy.

These correlation insights were confirmed by the analysis of performance metrics, where SVM and XGBoost demonstrated higher accuracy and recall compared to Logistic Regression. The confusion matrices for these models indicated better classification across project complexities, particularly for larger projects where prediction accuracy is crucial. This is particularly reflected in the violin plots of the model predictions, where XGBoost showed a concentrated prediction pattern around the median, and SVM displayed a broader prediction range, indicative of their capacity to manage diverse project scenarios and outcomes.

The specific interplay of these correlations with model outcomes provides a nuanced understanding of why SVM and XGBoost outperform Logistic Regression in this application. The strong correlation of 'Size KLOC' with 'ACT_EFFORT' directly contributes to the robust performance of SVM and XGBoost, both of which are optimized to exploit such relationships. The negligible correlation between 'PEMi' and 'ACT_EFFORT', however, highlights a critical limitation of Logistic Regression in its inability to account for complex, multi-dimensional influences on project effort.

Conclusion

Given the crucial role that accurate software effort and cost estimation have in handling project timelines, quality, and cost management, machine learning methods are essential for addressing the software industry's fundamental difficulties. The goal of this study was to improve software effort and cost estimation by applying supervised machine learning algorithms, particularly XGBoost and SVM, and comparing their performance against the baseline Logistic Regression model using the COCOMO approach. Our data indicate that XGBoost and SVM outperform Logistic Regression in terms of accuracy and recall, with SVM potentially achieving the greatest metrics.

However, the study found a few limitations. Due to dataset imbalance, the models appeared to be biased toward the majority class, which could lead to inaccurate performance measures. Furthermore, SVM and XGBoost require more processing costs than Logistic Regression, making them difficult to apply in real-time applications.

Future work should focus on overcoming these limitations by using advanced data augmentation techniques to balance the dataset and explore more computationally efficient algorithms. Furthermore, deep learning neural networks may improve prediction accuracy and help manage the complexity of software projects more efficiently.

Technology statement

The source of the data has been provided by the supervisor of this research project. Work on this paper did not involve collecting data from human participants or animals. All the figures belong to the authors. Part of the code has been adapted by the authors from the scikit-learn documentation website. In terms of writing, the authors used assistance

with the language of the paper. A generative language model (ChatGPT) was used to improve the author's original content, paraphrasing, spell-checking, and grammar. The tool Grammarly was used during the writing process to prevent grammar and spelling errors.

Contribution

Alexandra Antonica: Conceptualization, Methodology, Software, Data curation, Visualization, Writing - Original Draft, Review & Editing. **Pien Jansen:** Conceptualization, Validation, Writing Original Draft, Review & Editing **Barbara Koch:** Conceptualization, Methodology, Software, Data curation, Visualization, Writing - Original Draft, Review & Editing **Kaloyan Milchin:** Validation, Writing - Original Draft, Review & Editing. **Prakhar Tiwari:** Methodology, Formal Analysis, Visualization, Writing - Original Draft, Review & Editing.

All authors have read and agreed to the published version of this manuscript.

References

Akrami, H., Aydore, S., Leahy, R. M., & Joshi, A. A. (2020). Robust variational autoencoder for tabular data with beta divergence. *arXiv* preprint arXiv:2006.08204.

Akumba, B. O., Blamah, N., Agaji, I., & Ogala, E. (2023). Analysis of machine learning techniques used in software cost estimation: A review. *Quest Journals, Journal of Software Engineering and Simulation*, 9(4), 01-07. https://www.questjournals.org/jses/papers/Vol9-issue-4/09040107.pdf

Alauthman, M., al-Qerem, A., Alangari, S., Ali, A. M., Nabo, A., Aldweesh, A., Jebreen, I., Almomani, A., & Gupta, B. B. (2023). Machine Learning for Accurate Software Development Cost Estimation in Economically and Technically Limited Environments. *International Journal of Software Science and Computational Intelligence (IJSSCI)*, *15*(1), 1–24. DOI: 10.4018/IJSSCI.331753

Amaral, W., Rivero, L., Bráz, G., & Viana, D. (2019). Using machine learning techniques for effort estimation in software development. *XVIII Brazilian Symposium on Software Quality*. DOI: 10.1145/3364641.3364670

BaniMustafa, A. (2018). Predicting Software Effort Estimation Using Machine Learning Techniques. 2018 8th *International Conference on Computer Science and Information Technology (CSIT), 249–256*. DOI: 10.1109/CSIT.2018.8486222

Bardsiri, V. K., & Dorosti, M. (2016). An Improved COCOMO based Model to Estimate the Effort of Software Projects. J*ournal of Advances in Computer Engineering and Technology, 2(2), 11–22*. DOI: 10.48550/arXiv.2006.08204

H. I. H. Yusri, A. A. Ab Rahim, S. L. M. Hassan, I. S. A. Halim and N. E. Abdullah, "Water Quality Classification Using SVM And XGBoost Method," 2022 *IEEE 13th Control and System Graduate Research Colloquium (ICSGRC)*, Shah Alam, Malaysia, 2022, pp. 231-236, DOI: 10.1109/ICSGRC55096.2022.9845143.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. DOI: 10.1162/neco.1997.9.8.1735

Kumar, B. K., Bilgaiyan, S., & Mishra, B. S. P. (2023). Software effort estimation based on ensemble extreme gradient boosting algorithm and modified Jaya optimization algorithm. *International Journal of Computational Intelligence and Applications*. DOI: 10.1142/s1469026823500323

Machado, P., Fernandes, B., Novais, P. (2022). Benchmarking Data Augmentation Techniques for Tabular Data. In: Yin, H., Camacho, D., Tino, P. (eds) *Intelligent Data Engineering and Automated Learning – IDEAL 2022*. Lecture Notes in Computer Science, vol 13756. Springer, Cham. DOI: 10.1007/978-3-031-21753-1_11

Mahmood, Y., Kama, N., Azmi, A., Khan, A. S., & Ali, M. (2022). Software effort estimation accuracy prediction of machine learning techniques: A systematic performance evaluation. *Software: Practice and Experience*, 52(1), 39–65. DOI: 10.1002/spe.3009

Naser, M., & Alavi, A. H. (2020). Insights into Performance Fitness and Error Metrics for Machine Learning. *arXiv* (Cornell University). DOI: 10.48550/arXiv..2006.00887

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011b). Scikit-learn: Machine Learning in Python. *JMLR*. DOI: 10.5555/1953048.2078195

Ranković, N., Ranković, D., Ivanović, M., & Lazić, L. (2021). A new approach to software effort estimation using different artificial neural network architectures and Taguchi orthogonal arrays. *IEEE Access*, 9, 26926–26936. DOI: 10.1109/access.2021.3057807

Rao, N., Bolla, J. V., Mummana, S., Krishna, CH. M., Gandhi, O., & Stephen, J. (2022). *OLCE: Optimized Learning-based Cost Estimation for Global Software Project*. DOI: 10.21203/rs.3.rs-2024296/v1

Shah, R., Shah, V., Nair, A. R., Vyas, T., Desai, S., & Degadwala, S. (2022). Software Effort Estimation using Machine Learning Algorithms. *2022 6th International Conference on Electronics, Communication and Aerospace Technology*, 1–8. DOI: 10.1109/ICECA55336.2022.10009346

Sousa, A. O., Veloso, D. T., Gonçalves, H. M., Faria, J. P., Mendes-Moreira, J., Graça, R., Gomes, D., Castro, R. N., & Henriques, P. C. (2023). Applying Machine Learning to Estimate the Effort and Duration of Individual Tasks in Software Projects. *IEEE Access*, *11*, 89933–89946. DOI: 10.1109/ACCESS.2023.3307310

Suherman, I. C., Sarno, R., & Sholiq. (2020). Implementation of Random Forest Regression for COCOMO II Effort Estimation. *2020 International Seminar on Application for Technology of Information*

and Communication (iSemantic), 476–481. DOI: 10.1109/iSemantic50169.2020.9234269